

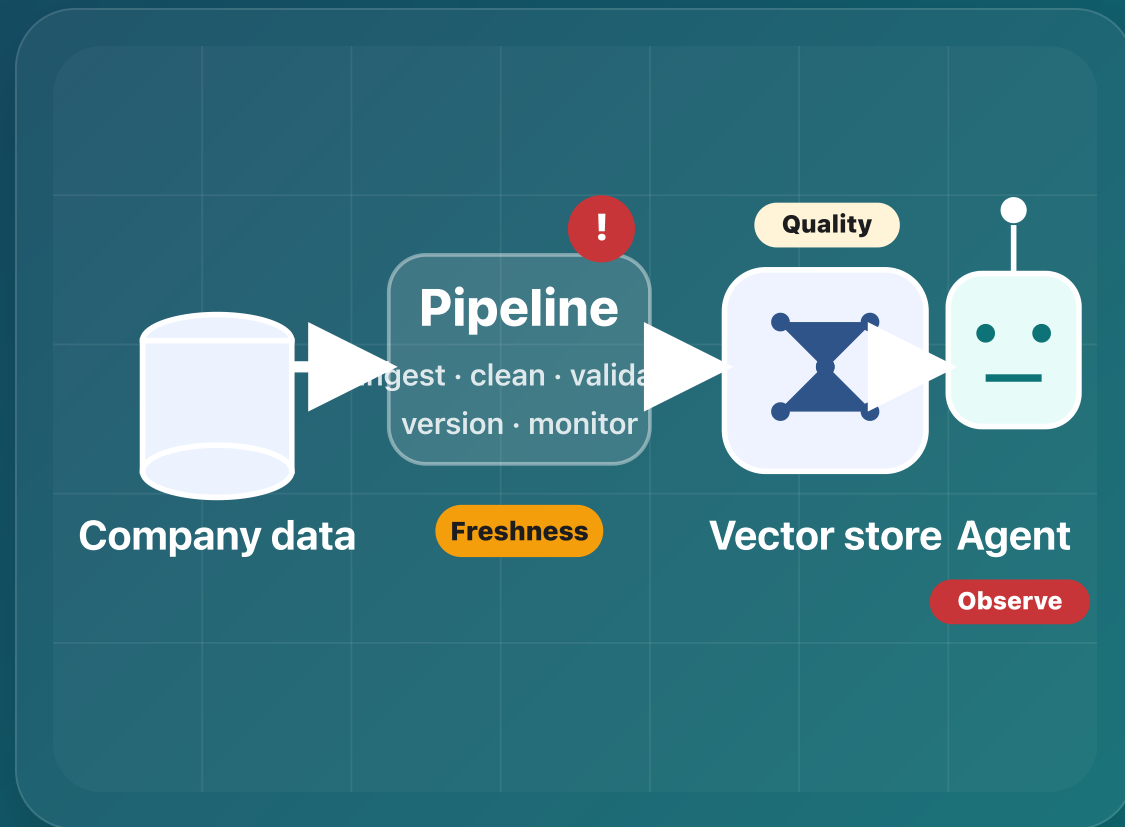
Data Pipeline & Data Observability

Ngày 10 · Garbage in → garbage out — fix thế nào?

Từ data thô đến agent đáng tin

ETL + Data Quality + Observability

Nối trực tiếp với RAG / multi-agent





[Tên giảng viên]

[Chức danh / vai trò]

- Audience: học viên đã có RAG / multi-agent artifact
- Focus: data pipeline, data quality, observability
- Case study: trợ lý nội bộ CS + IT Helpdesk
- Goal: detect issue trước khi user thấy câu trả lời sai

Topic: Data Pipeline & Data Observability

Data từ database công ty đột nhiên sai — agent hallucinate. Bạn có biết không?

Nếu chỉ biết sau khi user phàn nàn thì đã quá muộn

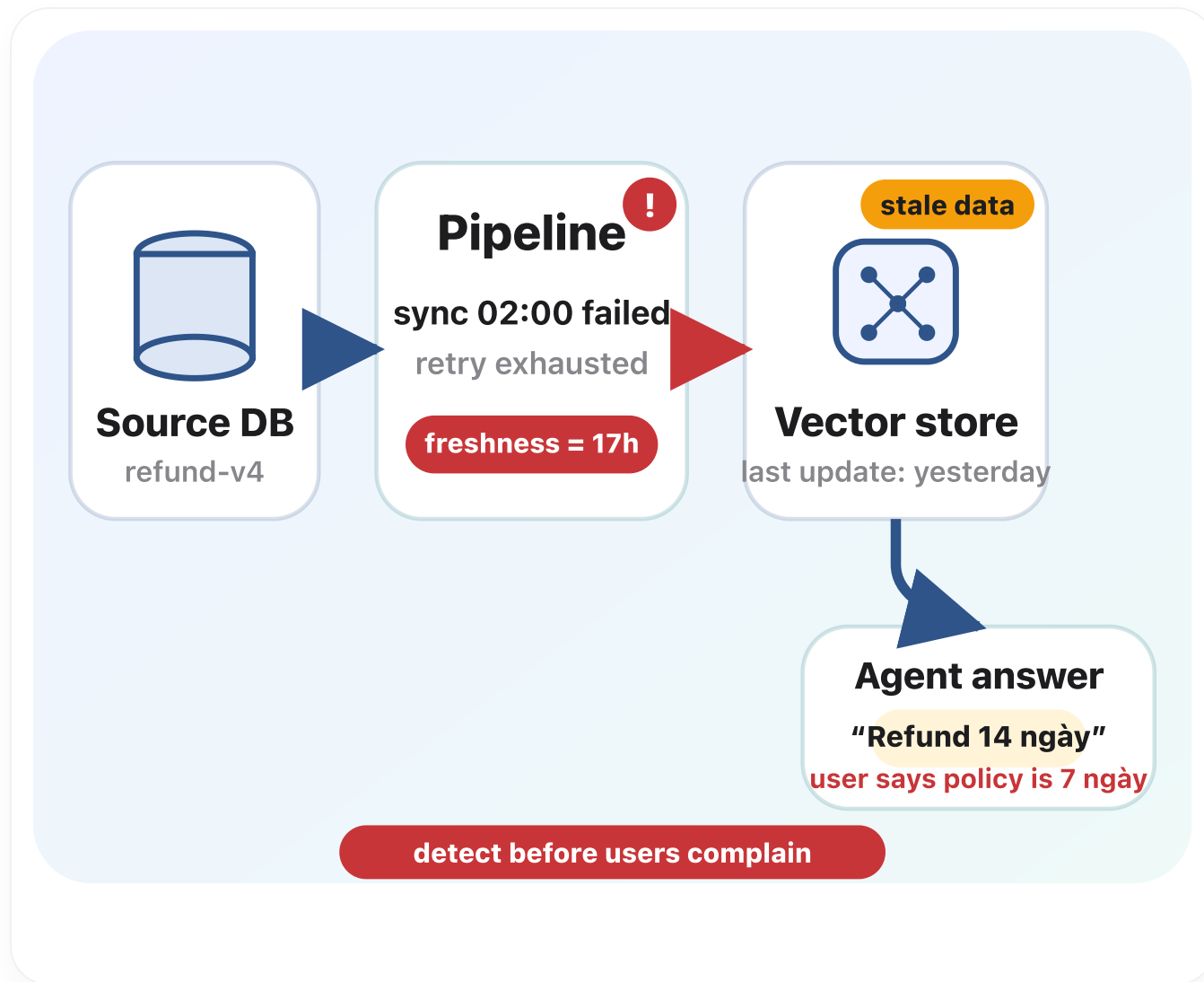
Mở bài vận hành

Agent mạnh vẫn trả lời sai nếu data đầu vào bản hoặc stale.

Observability = phát hiện vấn đề trước khi người dùng kêu

Garbage in → garbage out là bài toán hệ thống

Đừng debug model trước khi debug pipeline



Mục tiêu học tập & đầu ra cuối ngày

Học pipeline, quality, observability; build ETL + quality report + before/after comparison

Learn · 4h

ETL / ELT · batch / stream

Ingestion · cleaning · contract

Observability · expectation suite

Build · 4h

ETL pipeline chạy được

Freshness monitor + quality checks

Before / after ảnh hưởng lên agent

09:00–13:00

Lý thuyết + activity



14:00–18:00

Lab + deliverable

etl_pipeline.py

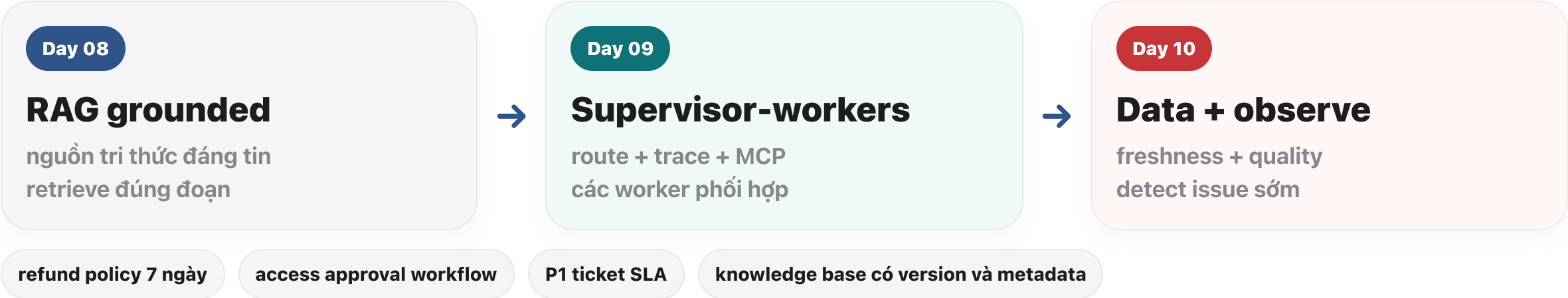
quality report

before_after_eval

runbook

Nhắc lại mạch 3 ngày: cùng một artifact được nâng cấp

Trợ lý nội bộ cho khối CS + IT Helpdesk



Triệu chứng agent ↔ tầng data bị lỗi

Đọc "sai" đúng tầng trước khi sửa prompt hay model

Trả lời đúng fact nhưng cũ

freshness · publish · cache vector

Trả lời lẫn lộn nguồn / chunk

dedupe · version metadata · chunk boundary

Trả lời "tự bịa" có vẻ hợp lý

missing grounding + retrieval gap — vẫn kiểm data trước

Chỉ một loại ticket sai

OCR/API cụ thể — isolate theo lineage nguồn

Đột biến sau deploy pipeline

schema/parser change — so sánh distribution trước/sau

Nguyên tắc

đo freshness + volume trước khi mở notebook fine-tune

Ai làm gì? (bản tối giản RACI)

Tránh “AI engineer ôm trọn data platform” không có ranh giới

Vai trò	Trách nhiệm chính	Artifact
AI / Applied	data contract cho corpus agent · eval before/after	data_contract.md · golden questions
Data Eng	ingest robust · modeling · pipeline SLA	pipeline_architecture.md · lineage
SRE / Platform	alert · on-call runbook · quota/secret	runbook.md · dashboards
Product / SME	định nghĩa “đúng” policy · sign-off version	source-of-truth link

Nhóm nhỏ lab: có thể một người đóng nhiều vai — nhưng vẫn ghi rõ owner trên từng nguồn.

SLA freshness nói bằng ngôn ngữ nghiệp vụ

Ví dụ: policy refund phải reflect trong agent ≤ 4 giờ sau khi PDF ký

Đo điểm cuối — thời điểm document “active” trên bảng cleaned hoặc vector index

Ngưỡng cảnh báo — warning ở 50% SLA, page ở 100% SLA

Phân loại nguồn — static PDF vs ticket stream không dùng chung một SLA

Hiển thị cho user — “Dữ liệu tri thức cập nhật tới: ...” giảm kỳ vọng sai

Checklist trước khi embed lần đầu

Copy vào docs nhóm lab nếu cần

Schema canonical + kiểu cột + primary key / natural key thống nhất

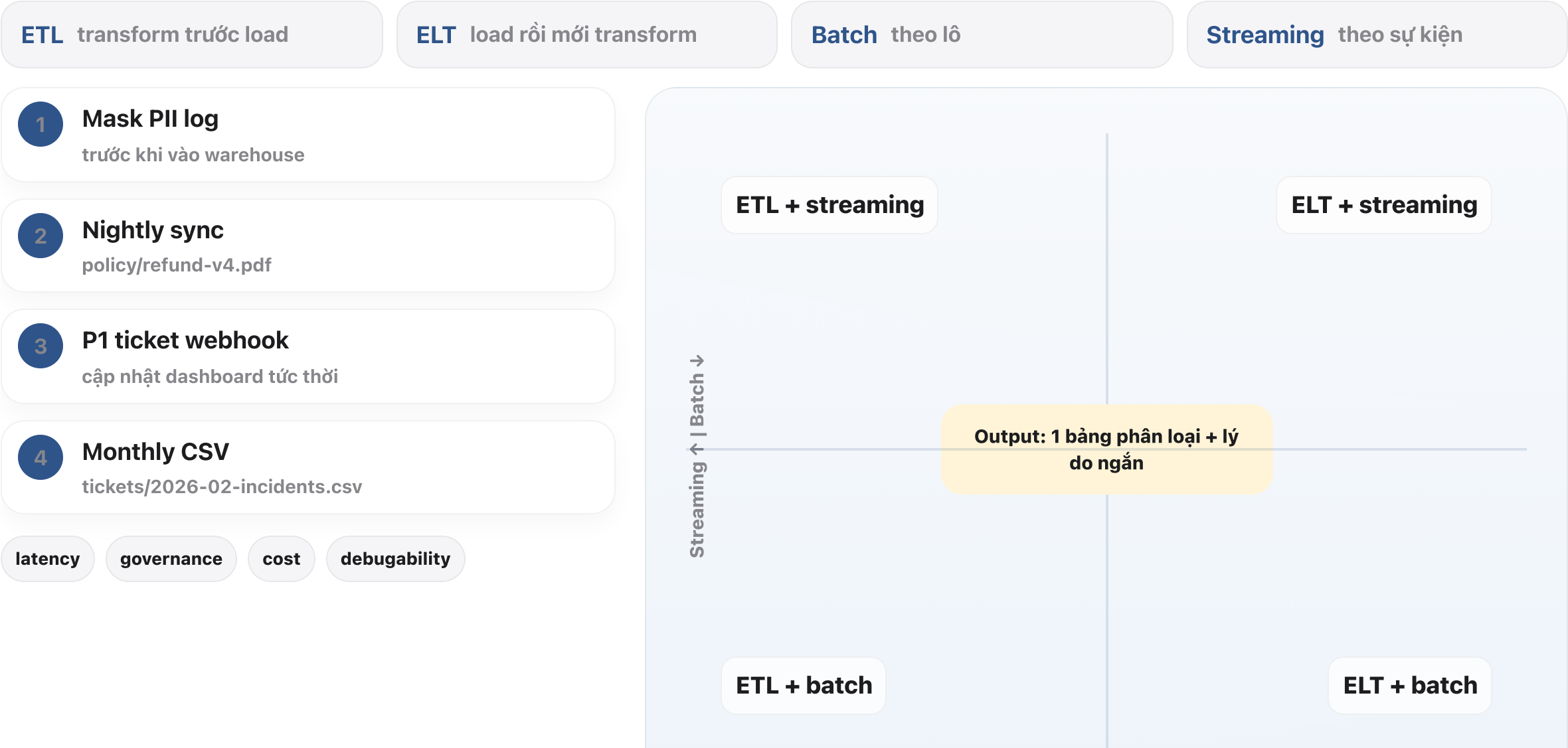
Encoding UTF-8 · test vài dòng tiếng Việt có dấu

Version / effective_date / source_uri ghi nhận được trong metadata chunk

Sample 20 row manual skim + 3 câu hỏi golden đặt trước cho agent

Hoạt động 1 — ETL hay ELT? Batch hay Streaming?

Phân loại 4 tình huống thực tế trước khi đi vào chi tiết



Hoạt động 1 – Gợi ý phân loại + lý do ngắn

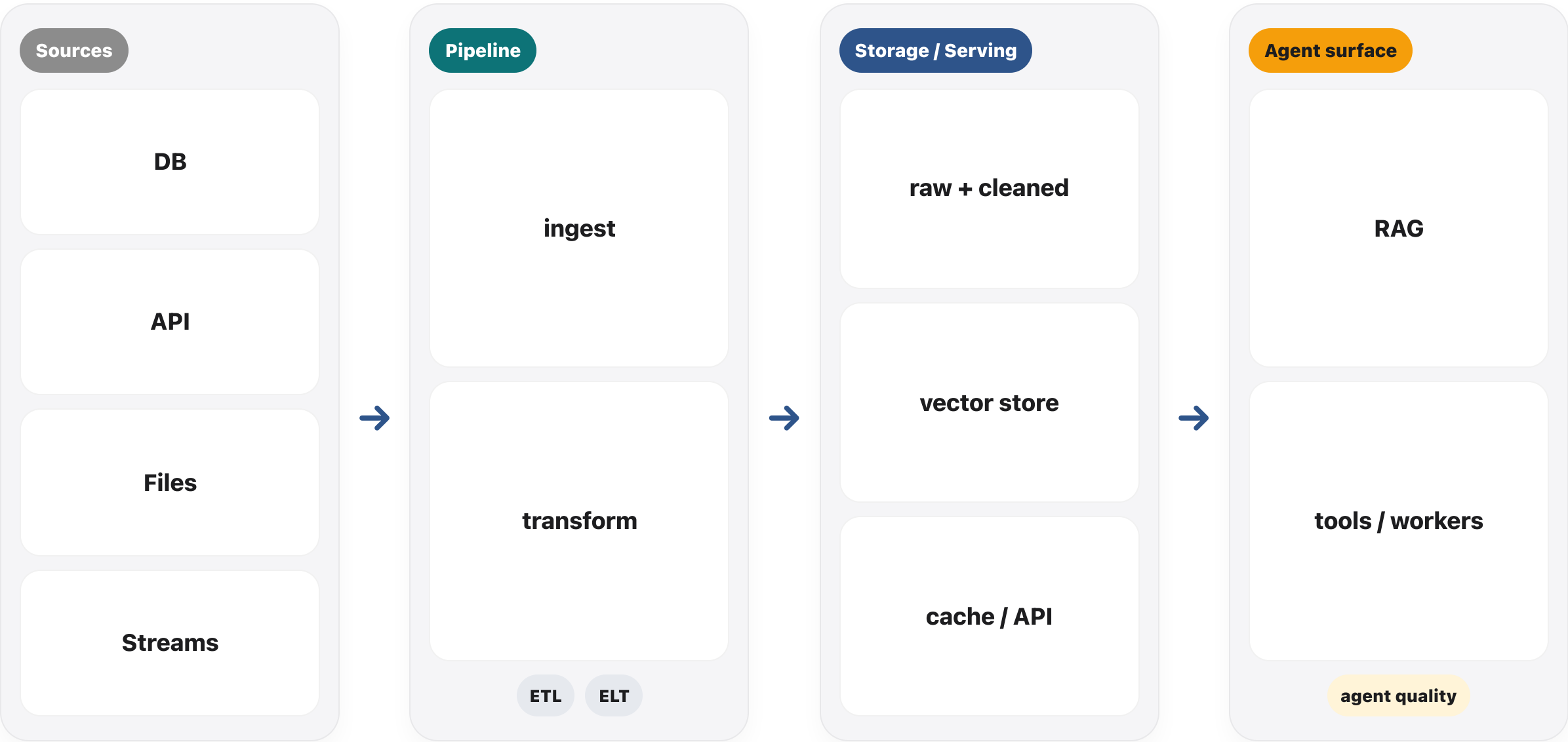
Đáp án không nhất thiết duy nhất; cần biện minh latency / governance / cost

#	Gợi ý ô	Lý do (1 dòng)
1 Mask PII log	ETL + streaming	phải che PII <i>trước</i> khi vào lake/warehouse → transform trước load; log tới liên tục
2 Nightly PDF	ETL + batch	khối lượng ổn định theo lịch; extract/chuẩn hoá text & metadata trước khi nạp staging
3 P1 webhook	ELT + streaming	ghi event/raw nhanh; aggregate/dashboard transform sau trong warehouse
4 Monthly CSV	ELT + batch	load file định kỳ vào lake trước; SQL/Notebook transform phía sau (debug dễ, chi phí hợp lý)

Nếu nhóm chọn khác: hợp lệ nếu nêu rõ trade-off (ví dụ streaming ELT nhưng thêm bước mask sớm).

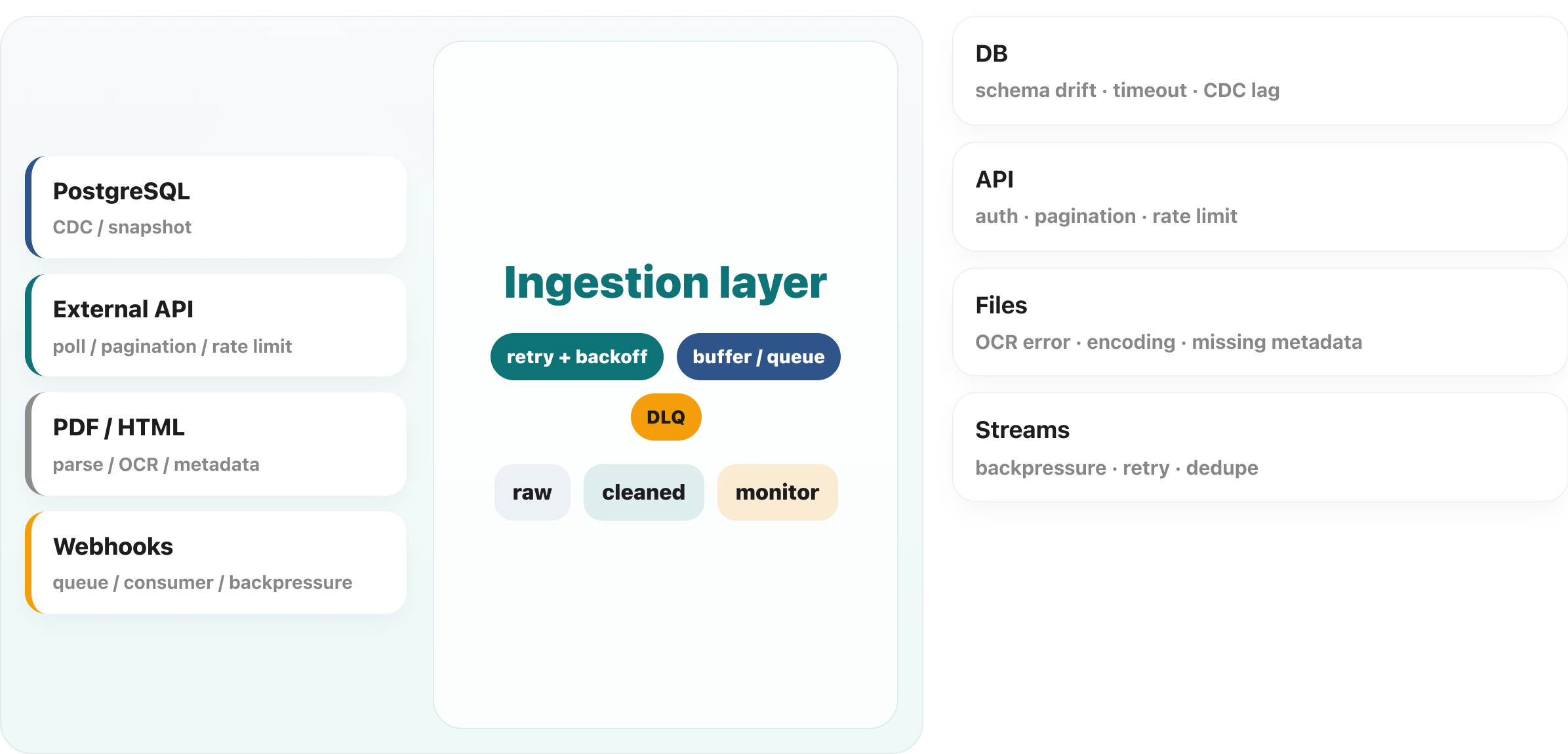
Data pipeline là gì trong AI stack?

Sources → Pipeline → Storage → Serving → Agent



Ingestion từ nhiều nguồn: DB, API, files, streams

Không chỉ kéo data về; phải xử lý CDC, rate limit, backpressure, retry



Nhắc nhanh trước khi vẽ source map

Nguồn nào · Hỏng kiểu gì · Đo cái gì

Source

PostgreSQL
External API
PDF / HTML

Failure

timeout
rate limit
OCR / schema drift

Monitor

freshness
volume
schema / lineage

Output template: source → method → check → owner

CDC vs snapshot: khi nào dùng gì?

Database là nguồn hay thay đổi nhất cho policy nội bộ

Snapshot / batch query

full hoặc incremental theo watermark updated_at — đơn giản, dễ hiểu

CDC (log-based)

độ trễ thấp, ít miss update nếu có slot WAL

Trade-off

CDC phức tạp hơn vận hành; snapshot dễ “đọc quá nhiều”

Monitor snapshot

watermark stall · duration query · rows/sec

Monitor CDC

replication lag · consumer offset · poison message

Agent note

CDC trễ vài phút vẫn có thể làm policy “cũ” với ticket P1

API ingestion — pagination, cursor, quota

Thiết kế để không bị 429 làm “đứng hình” pipeline

Token bucket / rate limit header — respect Retry-After, exponential backoff + jitter

Pagination — cursor ổn định hơn offset khi data đang thay đổi

Checkpoint — lưu last_synced_id idempotent; rerun không double-fetch toàn bộ

Partial page failure — ghi page đã fail vào DLQ để replay có chủ đích

Files (PDF/HTML) — parse, hash, version

Tránh embed nhầm bản policy cũ chỉ vì tên file giống

Content hash — sha256 của bytes; đổi nhẹ metadata không làm re-embed oan

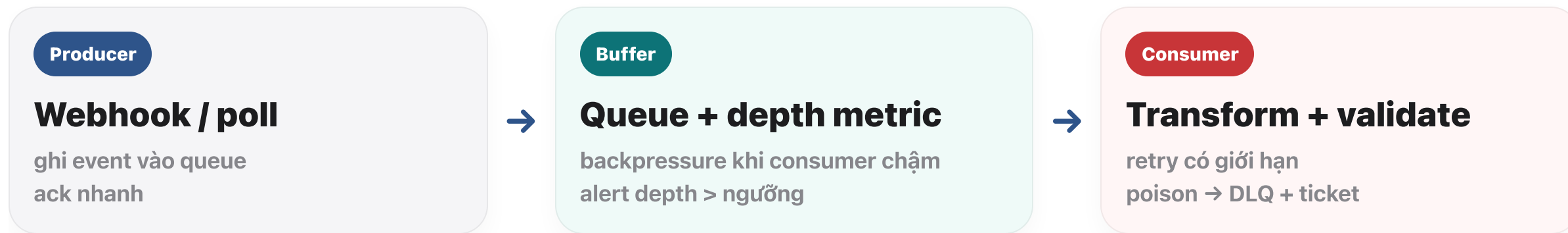
Logical version — `policy_v4` gắn metadata cho vector filter

OCR path — confidence score; thấp → flag human review trước publish

Chunk boundary — heading-aware chunk giảm câu trả lời “cắt nửa điều khoản”

Queue · backpressure · DLQ (một sơ đồ tư duy)

Tách ingest nhanh khỏi transform chậm



DLQ không phải “thất bại”; là điểm quan sát để không làm mất sự kiện.

Hoạt động 2 — Mapping Ingestion Risks

Thiết kế ingestion plan cho use case CS & IT Helpdesk

Case Study: CS & IT Support

- **PostgreSQL**: Thông tin khách hàng & lịch sử ticket.
- **Jira API**: Trạng thái xử lý lỗi thời gian thực.
- **PDF Docs**: Quy trình (SOP) & Policy bảo mật.

Nhiệm vụ (3 Bước)

1. **Identify**: Xác định logic kéo data cho từng nguồn.
2. **Foresee**: Đánh dấu các "điểm đứt" (failures).
3. **Protect**: Đề xuất cách monitor để phát hiện sớm.

schema drift

OCR error

timeout

rate limit

Pipeline Funnel: Từ Nguồn đến Agent



Source	Risk (What breaks?)	Monitor (How to detect?)
PostgreSQL	CDC Lag / Schema drift	?
Jira API	429 Rate Limit / Auth	?
PDF SOPs	Bad OCR / Formatting	?

Goal: Điền các dấu "?" và đánh dấu rủi ro vào sơ đồ.

Hoạt động 2 — Source map & bảng ingestion (ví dụ)

Use case: CS/IT Helpdesk — PostgreSQL + CRM API + PDF SOP

2 rủi ro chính

- 1) API CRM rate limit 429 làm mất nhịp sync.
- 2) PDF scan OCR confidence thấp → text rác vào RAG.

1 monitor ưu tiên

freshness_sla trên bảng kb_documents (end-to-end tới vector publish).

Source	Method	Check	Owner
PostgreSQL tickets	CDC / watermark updated_at	freshness + replication lag	Data Eng
CRM API	poll + cursor checkpoint	429 rate + checkpoint stall	AI Eng
PDF SOP	parse + OCR confidence	% row flag review > 5%	AI Eng
Buffer	queue (Redis/SQS)	depth + DLQ count	SRE

Sơ đồ một dòng: 3 nguồn → queue → ingest worker → raw → clean → validate → embed → publish

Transform: làm sạch và chuẩn hoá trước khi agent dùng được

Missing, duplicates, wrong format, encoding, schema validation

Raw sample

doc_id	title	date	content
12	Refund policy	07/02/26	7 ngày
12	Refund policy	2026-02-07	7 ngày
44	Access approve	NULL	quy trình...
58	P1 SLA	2026/02/10	2h – lỗi mã hoá ❓

dedupe

date parse

unicode

schema check

Cleaned sample

id	version	effective_date	status
12	v4	2026-02-07	active
44	v2	missing → flag	review
58	v1	2026-02-10	active

Transform = code + contract, không sửa tay ngay trước khi demo.

Hoạt động 3 — Dirty data repair

Từ một CSV lỗi, đề xuất cleaning rules trước khi embed

doc_id	source	date	content	notes
12	refund-v4.pdf	2026-02-07	Refund 7 ngày	ok
12	refund-v4.pdf	07/02/26	Refund 7 ngày	duplicate
44	access-control-sop.md	NULL	Approve by manager	missing date
58	sla-p1-2026.pdf	2026/02/10	SLA = 2h ⚠	encoding
67	faq/account-recovery.html	2026-02-11		empty content

Clean

trim whitespace
parse date
normalize unicode

Reject

empty content
duplicate primary key

Flag

missing date
manual review

Hoạt động 3 — Bộ cleaning rules (≥ 5 rule)

Áp dụng trực tiếp lên bảng dirty trên slide hoạt động

Clean / normalize

1. Trim whitespace cột content và notes.
2. Parse date đã định dạng $\rightarrow$ YYYY-MM-DD UTC.
3. Unicode NFC + thay thế ký tự thay thế cho "".

Reject cứng

4. Drop row nếu content rỗng sau trim.
5. Drop duplicate theo khóa (doc_id, source, normalized_content_hash) — giữ bản mới nhất theo date parse được.

Flag / quarantine

6. Nếu date NULL nhưng content có $\rightarrow$ flag review_missing_date, không embed cho đến khi SME điền.
7. Log mọi drop/flag vào quarantine.csv kèm run_id.

Data quality as code

6 dimensions + Great Expectations / expectation suite

Completeness

content not null

Accuracy

policy date đúng

Consistency

same key, same format

Timeliness

freshness theo SLA

Validity

schema + contract hợp lệ

Uniqueness

không duplicate record

Expectation suite

```
batch.expect_column_values_to_not_be_null("content")
batch.expect_column_values_to_be_unique("doc_id")
batch.expect_column_values_to_match_regex(
    "effective_date", r"^\d{{4}}-\d{{2}}-\d{{2}}$"
)
if expectations_fail:
    raise PipelineHalt("bad data before agent")
```

fail pipeline if expectations fail

5 pillars of data observability

Freshness, distribution, volume, schema, lineage

Data observability

Freshness

17h

Volume

-28%

Schema

1 drift

Distribution

stable

Freshness timeline

02:00 failure



Lineage

DB → ingest → clean → embed → agent

trace ngược được step fail

Freshness

Data có cập nhật đúng nhịp không?

Distribution

Giá trị có lệch bất thường không?

Volume

Số record có rơi đột ngột không?

Schema

Cấu trúc có drift không?

Lineage

Biết lỗi đi từ nguồn nào tới output nào

Schema evolution: additive vs breaking

Giữ agent không “gãy” khi nguồn đổi cột

Additive (ưu tiên)

thêm cột optional · default null · version field

Quan sát

alert khi % null tăng đột biến sau deploy parser

Contract ở biên

parser ra canonical schema trước khi embed

Rollback

giữ 2 version schema song song trong staging

Breaking change

đổi kiểu / đổi ý nghĩa cột → cần migration + backfill có run_id

Agent impact

chunk boundary đổi → retrieval lệch dù “data vẫn có”

Mức độ nghiêm trọng expectation: warn · quarantine · halt

Không phải mọi lỗi đều dừng pipeline

Halt — duplicate primary key, encoding không recover, schema contract fail cứng

Quarantine — row đúng key nhưng content nghi ngờ → bảng quarantine + review

Warn + tiếp tục — missing optional field, typos trong metadata ít rủi ro

Policy — ghi rõ trong data_contract.md để agent không đọc nhầm nguồn “review”

Distribution & anomaly — ngoài “đủ record”

Phát hiện ingest “chạy nhưng sai”

Null rate

effective_date null % tăng sau đổi API

Length

content quá ngắn / quá dài bất thường

Cardinality

số doc_id unique giảm mạnh → dedupe lỗi

Entropy / language

mix ngôn ngữ hoặc OCR noise spike

Cross-field

end < start · version < predecessor

Time travel

future_effective_date > clock skew ngưỡng

SLI gợi ý cho tầng RAG / agent

Liên kết observability data với trải nghiệm user

Citation freshness — tuổi của chunk được trích dẫn trung bình / p95

Grounding rate — % câu trả lời có trích nguồn hợp lệ (nếu thiết kế yêu cầu)

Retrieval hit@k — trên tập câu hỏi regression (golden set nhỏ)

Latency pipeline — ingest→publish p95; tách biệt latency LLM

Khi SLI xấu, quay lại 5 pillars trước khi tin vào “model bị lỗi”.

Khi agent trả lời cũ, đừng debug model trước.

Freshness

Volume

Schema

Lineage

Root cause

Hoạt động 4 — Incident triage

Agent trả lời sai vì dùng data cũ: debug từ dashboard nào trước?

P1 · 09:12

User phàn nàn agent trả lời policy cũ

Chọn thứ tự check dashboard trước khi mở code.

Output

1 triage flow + metric cần xem ở từng bước

1 · Detect

freshness có trễ không?



2 · Isolate

volume có rơi / spike không?



3 · Validate

schema drift hay parse error?



4 · Trace lineage

nguồn nào → step nào fail?



5 · Fix + rerun

idempotent rerun + verify answer

Hoạt động 4 — Ví dụ triage flow + metric từng bước

Học viên có thể đối chiếu hoặc chỉnh theo hệ thực tế

1 · Detect

Metric: freshness_hours trên bảng kb_documents = 17h (ngưỡng 4h) → **giả thuyết:** sync trễ hoặc fail



2 · Isolate

Metric: rows_ingested giảm -28% so với 7 ngày trước; embed_task success nhưng input nhỏ bất thường



3 · Validate

Check: schema contract PDF parser — không drift; **log:** ingest_02:00 retry exhausted



4 · Lineage

refund-v4.pdf → ingest job → clean → chunk → vector policy_v4 — điểm đứt: ingest (không phải model)



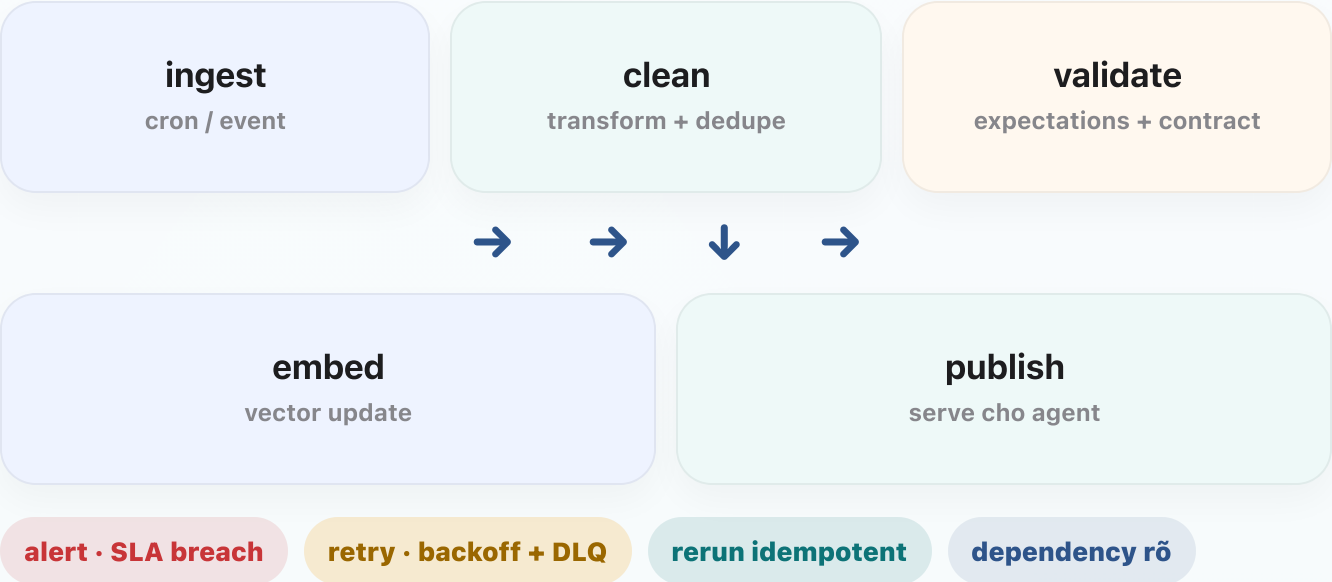
5 · Fix + verify

rerun idempotent ingest + publish; verify: freshness < 4h + agent trả lời "7 ngày" khớp policy mới

Orchestration, scheduling, retry và idempotency

Batch pipeline tốt không chỉ chạy theo cron; nó còn phải re-run an toàn

DAG tối giản: batch pipeline chạy lại an toàn



Schedule

cron, event, backfill

Retry

backoff · DLQ · alert

Visibility

DAG + dependency + SLA breach

Idempotency

rerun 2 lần không duplicate vector store

Triage có timebox — tránh “mở hết dashboard”

Áp dụng khi user đang chờ (P1 / policy)

0–5 phút

Freshness + last_success_run — có trễ SLA không?

5–12 phút

Volume + error rate step — step nào “im lặng” hoặc spike?

12–20 phút

Schema/contract + lineage tới bảng/file nguồn

Nếu chưa ra root cause

chuyển sang “giảm thiểu”: rollback publish hoặc banner “data đang cập nhật”

Ghi incident note

metric đã xem + kết luận từng bước (dù fail)

Sau khi fix

verify bằng cùng run_id trên cleaned + embed + 1 query agent

Idempotency — pattern thực dụng

Rerun an toàn cho embed và warehouse

Natural key — upsert theo (doc_id, version) thay vì random UUID mỗi lần

Partition theo ngày — ghi đè partition cùng run_id thay vì append mù

Delete-then-insert trong transaction cho phạm vi doc_id (cẩn thận lock)

Vector store — policy “replace collection staging → alias swap” giảm partial read

Orchestrator = dependency graph + visibility

Airflow / Prefect / Dagster — cùng một câu hỏi thiết kế

Trigger

cron vs event vs upstream success

Retry policy

max_attempt, backoff, retry_only_failed_task

SLA / alert

task duration, queue depth, missed schedule

Data awareness

sensor đợi file/partition có mặt

Observability

task log + lineage tag run_id

Backfill

chạy lại range ngày không phá production alias

Sau incident: cập nhật runbook (template)

Biến sự cố thành tài sản cho lần sau

Symptom

user thấy gì · agent trả lời sai kiểu gì

Detection

metric/dashboard nào đã báo (hoặc chưa → gap)

Diagnosis

root cause · step pipeline · bảng/file

Mitigation

rollback / rerun / feature flag

Prevention

expectation mới · alert mới · owner

Hands-on 10 — ETL + Quality + Monitoring

4 giờ thực hành: raw → cleaned → embedded, inject corruption, đo ảnh hưởng lên agent



Sprint 1
ingest raw → cleaned

Sprint 2
cleaned → embedded

Sprint 3
inject corruption + compare

Sprint 4
expectation suite + monitor + docs

Before / after answer quality

dirty

cleaned

Deliverables nhóm

etl_pipeline.py

transform/cleaning rules

expectation suite + freshness monitoring

quality report + before/after eval

pipeline_architecture.md · data_contract.md · runbook.md

ETL pipeline

45%

Documentation

25%

Quality evidence

20%

Individual report

10%

Hướng dẫn thực hành 1 — setup & Sprint 1-2

Mục tiêu: có cleaned data, log trước/sau và pipeline end-to-end

Folder tree tối thiểu

```
etl_pipeline.py transform/ cleaning_rules.py  
quality/ expectations.py monitoring/  
freshness_check.py docs/ pipeline_architecture.md
```

Run order: ingest → clean → validate → embed

- 1 Map source và chuẩn hoá schema đầu vào
- 2 Log số record trước / sau cleaning
- 3 Đặt run_id hoặc timestamp cho pipeline
- 4 Đến cuối Sprint 2: raw → cleaned → embedded chạy end-to-end

Expected log

```
raw_records=128  
cleaned_records=121  
dropped_duplicates=5  
flagged_missing_date=2  
run_id=2026-02-10T14:35
```

Hướng dẫn thực hành 2 — Sprint 3-4 & evidence

Mục tiêu: inject corruption, đo ảnh hưởng và nộp report có bằng chứng

missing

duplicate

wrong format

encoding

Before

answer stale
retrieval lệch
chunk lỗi

After

answer đúng version
trace rõ run_id
freshness pass

freshness_sla

PASS

schema_contract

PASS

duplicate_key

2 flagged

Evidence cần nộp

before_after_eval.csv hoặc ảnh dashboard

quality report

runbook / monitor screenshot

báo cáo cá nhân ngắn gọn

Khung báo cáo cá nhân

1. Phần tôi phụ trách
2. Một lỗi hoặc quyết định kỹ thuật chính
3. Bằng chứng trước / sau
4. Một cải tiến tiếp theo

Definition of Done cho pipeline ngày hôm nay

Dùng làm checklist trước khi nộp bài

Có **run_id** hoặc **timestamp** gắn mọi artifact (log, cleaned, embed)

Có log **số record** raw → cleaned → embedded (drop/flag có giải thích)

Expectation suite chạy được; fail thì pipeline **dừng có kiểm soát**

Có ít nhất 1 bằng chứng **before/after** ảnh hưởng tới câu trả lời agent

“Chạy được trên máy giảng viên” = script + README + lệnh chạy một dòng.

Peer review nhanh giữa các nhóm

10 phút đổi bàn — giảm lỗi cấu hình trước khi chấm

Người review hỏi

“Rerun 2 lần có duplicate vector không?”

Người review hỏi

“Freshness đo ở bước nào — ingest hay publish?”

Người review hỏi

“Record bị flag đi đâu — quarantine hay vẫn embed?”

Nhóm trả lời tối thiểu

chỉ vào file + dòng log minh chứng

Nếu không rõ

ghi vào runbook.md là “gap” cần bổ sung

Kết quả

mỗi nhóm có 3 bullet feedback đã xử lý hoặc đã ghi nhận

Git & artifact hygiene cho data pipeline

Nhỏ nhưng tránh nộp bài “chỉ chạy trên máy tôi”

.env.example — liệt kê biến; không commit secret

Pin version thư viện (requirements.txt / pyproject) có version cụ thể

Sample data trong data/sample/ đủ nhỏ để chấm nhanh

docs/ — pipeline_architecture + data_contract + runbook cùng commit cuối

Kịch bản demo 3 phút trước lớp

Cấu trúc cố định: bệnh → đo → sửa → chứng minh

0:00–0:45

Giả thích use case + nguồn (DB/API/file)



0:45–1:30

Inject corruption → chỉ freshness/volume trên dashboard hoặc log



1:30–2:30

Rerun pipeline idempotent → expectation pass



2:30–3:00

So sánh câu agent trước/sau + run_id khớp evidence

Dữ liệu sạch + quan sát tốt = nền cho AI vận hành bền

Tổng kết key takeaways và chuẩn bị cho lớp safety / guardrails tiếp theo

- 1

60–80% effort thực tế nằm ở data work
- 2

Quality checks phải chạy như code
- 3

Observability giúp biết lỗi trước user
- 4

Bước tiếp theo: safety / guardrails / runbook

Day 08

RAG grounded

nguồn tri thức đáng tin

Day 09

Supervisor-workers

route + trace + MCP

Day 10

Data + observe

nuôi và giám sát đúng

Day 11

Guardrails / Safety

giữ agent an toàn

Q&A

Nếu chỉ được giữ 1 dashboard cho hệ AI của mình, bạn giữ dashboard nào — agent answer hay data observability?